Software Engineering

Course 6 – 31 March 2025

adiftene@info.uaic.ro

Content

- From previous courses…
- SOLID Principles
- Design Patterns
  - Definitions
  - Elements
  - Example
  - Classification
- JUnit Testing
  - Netbeans (Example 1)
  - Eclipse (Example 2)

From previous courses

- Stages of Program Development
- Requirements engineering
- UML Diagrams
- GRASP
- SOLID

R – GRASP

- Principles, responsabilities
- Information Expert
- Creator
- Low Coupling
- High Cohesion
- Controller

R – SOLID and Other Principles

- SOLID Principles
  - SRP – Single Responsibility Principle
  - OCP – Open/Closed Principle
  - LSP – Liskov Substitution Principle
  - ISP – Interface Segregation Principle
  - DIP – Dependency Inversion Principle
- DRY – Don't Repeat Yourself
- YAGNI – You Aren't Gonna Need It
- KISS – Keep It Simple, Stupid

Content

- From previous courses…
- Design Patterns
  - Definitions
  - Elements
  - Example
  - Classification
- JUnit Testing
  - Netbeans (Example 1)
  - Eclipse (Example 2)

Design Patterns – Why?

- **If a problem occurs over and over again**, a solution to that problem has been used effectively (solution = pattern)

- When you make a design, you **should know the names of some common solutions**. Learning design patterns is good for people to **communicate each other effectively**

Design Patterns – Definitions

- "Design patterns capture solutions that have developed and evolved over time" (GOF – ***Gang-Of-Four*** (because of the four authors who wrote it), *Design Patterns: Elements of Reusable Object-Oriented Software)*

- **In software engineering (or computer science), a design pattern is a general repeatable solution to a commonly occurring problem in software design**

- The **design patterns** are language-independent strategies for solving common object-oriented design problems

Gang of Four

- Initial was the name given to a leftist political faction composed of four Chinese Communist party officials
- The name of the book (“Design Patterns: Elements of Reusable Object-Oriented Software”) is too long for e-mail, so “book by the gang of four” became a shorthand name for it
- That got shortened to "**GOF book**“. Authors are: *Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides*
- The **design patterns** in their book are *descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context*

Design Patterns – Elements

1. Pattern name
2. Problem
3. Solution
4. Consequences

Design Patterns – Pattern name

- A handle **used to describe a design problem**, its solutions, and consequences in a word or two
- Naming a pattern immediately increases our **design vocabulary**. It lets us design at a higher level of abstraction
- Having a **vocabulary** for patterns lets us talk about them with our colleagues, in our documentation
- Finding good names has been one of the hardest parts of developing our catalog

Design Patterns – Problem

- Describes **when** to apply the pattern. It explains the problem and its **context**
- It might describe specific design problems such as how to represent **algorithms** as objects
- It might describe **class** or **object** structures that are symptomatic of an inflexible design
- Sometimes the problem will include a **list of conditions** that must be met before it makes sense to apply the pattern

Design Patterns – Solution

- Describes the elements that make up the **design, their relationships, responsibilities,** and **collaborations**
- The solution **doesn't describe a particular concrete design or implementation**, because a pattern is like a template that can be applied in many different situations
- Instead, the pattern provides an **abstract description of a design problem** and **how** a general arrangement of elements (classes and objects in our case) **solves it**

Design Patterns – Consequences

- Are the results and trade-offs of applying the pattern
- They are critical for **evaluating design alternatives** and for **understanding the costs and benefits** of applying the pattern
- The consequences for software often concern **space** and **time trade-offs**, they can address **language and implementation issues** as well
- Include its impact on a system's **flexibility, extensibility, or portability**
- Listing these consequences explicitly helps you **understand and evaluate** them

Example of (Micro) pattern

- **Pattern name**: Initialization
- **Problem**: It is important for some code sequence to be executed only once at the beginning of the execution of the program.
- **Solution**: The solution is to use a static variable that holds information on whether or not the code sequence has been executed.
- **Consequences**: The solution requires the language to have a static variable that can be allocated storage at the beginning of the execution, initialized prior to the execution and remain allocated until the program termination.

Describing Design Patterns 1

- **Pattern Name and Classification**
- **Intent** – the answer to question: *What does the design pattern do*?
- **Also Known As**
- **Motivation** – A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem
- **Applicability** – *What are the situations in which the design pattern can be applied*? *How can you recognize these situations*?
- **Related Patterns**

Describing Design Patterns 2

- **Structure** – A graphical representation of the classes in the pattern
- **Participants** – The classes and/or objects participating in the design pattern and their responsibilities
- **Collaborations** – How the participants collaborate to carry out their responsibilities
- **Consequences** – *How does the pattern support its objectives*?
- **Implementation** – *What techniques should you be aware of when implementing the pattern*?
- **Sample Code**
- **Known Uses** – Examples of the pattern found in real systems

Design Patterns – Classification

- **Creational patterns**
- **Structural patterns**
- **Behavioral patterns**
- NOT in GOF: Fundamental, Partitioning, GRASP, GUI, Organizational Coding, Optimization Coding, Robustness Coding, Testing, Transactions, Distributed Architecture, Distributed Computing, Temporal, Database, Concurrency patterns

Creational Patterns

- **Abstract Factory** groups object factories that have a common theme
- **Builder** constructs complex objects by separating construction and representation
- **Factory Method** creates objects without specifying the exact class to create
- **Prototype** creates objects by cloning an existing object
- **Singleton** restricts object creation for a class to only one instance
- Not in GOF book: Lazy initialization, Object pool, Multiton, Resource acquisition (is initialization)

Structural Patterns

- **Adapter** allows classes with incompatible interfaces to work together
- **Bridge** decouples an abstraction from its implementation so that the two can vary independently
- **Composite** composes zero-or-more similar objects so that they can be manipulated as one object.
- **Decorator** dynamically adds/overrides behavior in an existing method of an object
- **Facade** provides a simplified interface to a large body of code
- **Flyweight** reduces the cost of creating and manipulating a large number of similar objects
- **Proxy** provides a placeholder for another object to control access, reduce cost, and reduce complexity

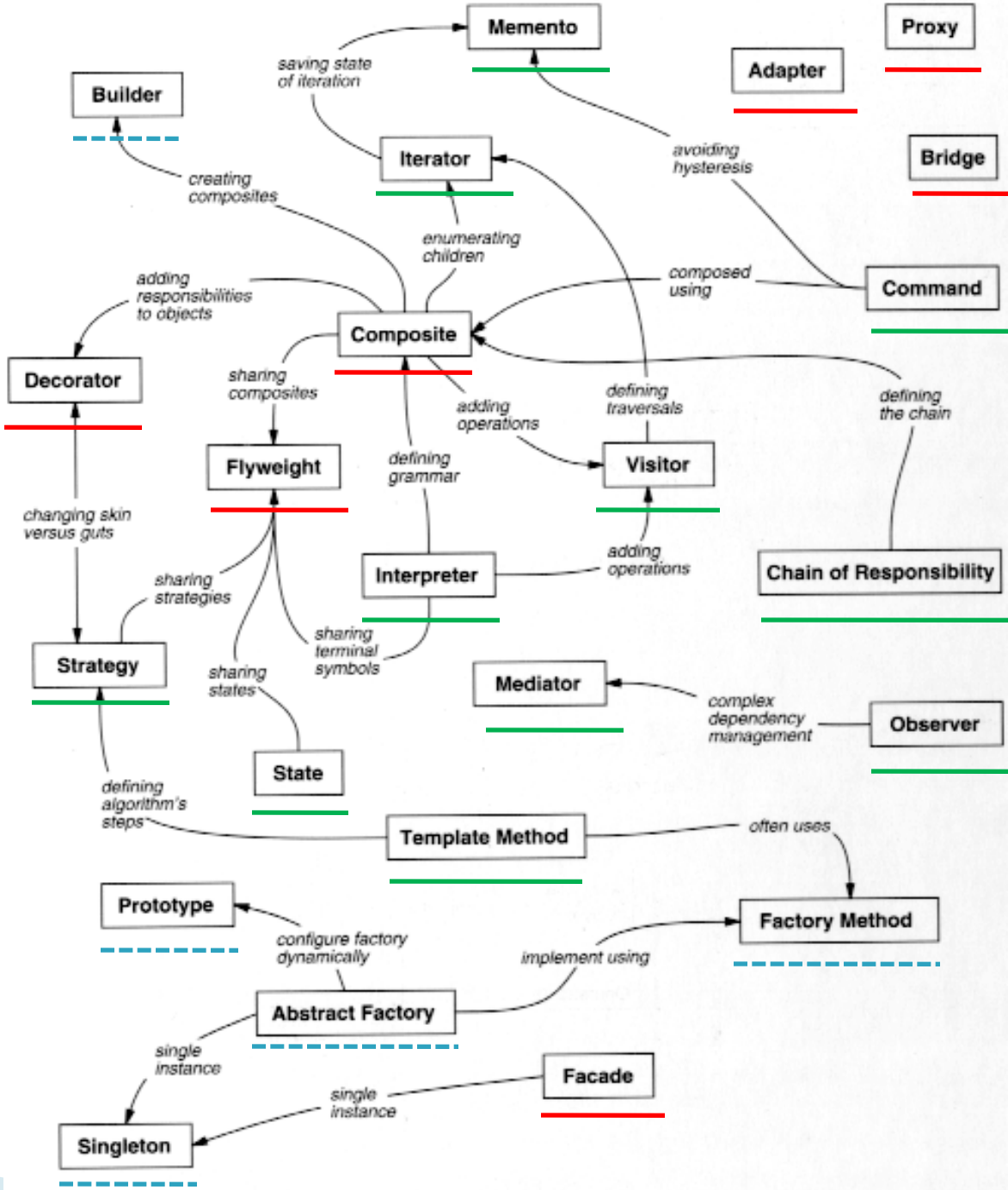
Behavioral patterns 1

- **Chain of responsibility** delegates commands to a chain of processing objects
- **Command** creates objects which encapsulate actions and parameters
- **Interpreter** implements a specialized language
- **Iterator** accesses the elements sequentially
- **Mediator** allows loose coupling between classes by being the only class that has detailed knowledge of their methods
- **Memento** provides the ability to restore an object to its previous state

Behavioral patterns 2

- **Observer** allows to observer objects to see an event
- **State** allows an object to alter its behavior when its internal state changes
- **Strategy** allows one of a family of algorithms to be selected on-the-fly at runtime
- **Template** defines an algorithm as an abstract class, allowing its subclasses to provide concrete behavior
- **Visitor** separates an algorithm from an object structure
- Not in GOF book: Null Object, Specification

- Patterns
  - Creational
  - Structural
  - Behavioral



Design Pattern Relationships

How to Select a Design Pattern?

- With more than 20 design patterns to choose from, it might be hard to find the one that addresses a particular design problem

- Approaches to finding the design pattern that's right for your problem:
  1. *Consider how design patterns solve design problems*
  2. *Scan Intent sections*
  3. *Study relationships between patterns*
  4. *Study patterns of like purpose (comparison)*
  5. *Examine a cause of redesign*
  6. *Consider what should be variable in your design*

How to Use a Design Pattern?

1. *Read the pattern once through for an overview*
2. *Go back and study the Structure*, Participants, and Collaborations sections
3. *Look at the Sample Code section to see a concrete example*
4. *Choose names for pattern participants that are meaningful in the application context*
5. *Define the classes*
6. *Define application-specific names for operations in the pattern*
7. *Implement the operations to carry out the responsibilities and collaborations in the pattern*

Content

- From previous courses…
- Design Patterns
  - Definitions
  - Elements
  - Example
  - Classification
- JUnit Testing
  - Netbeans (Example 1)
  - Eclipse (Example 2)

Unit Testing

- Testing a function, a program, a screen, a functionality
- It is done by programmers
- By default
- The results must be documented
- Input and Output simulators are used

Unit Testing – Example 1 (1)

Suma - NetBeans IDE 6.7.1

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects | Files | Services

- Suma
  - Source Packages
    - suma
      - Main.java
  - Test Packages
  - Libraries
  - Test Libraries

suma - Navigator

Members View

- Main
  - main(String[] args)
  - suma(int a, int b) : int

Start Page | Main.java *

```
 * and open the template in the editor.
 */

package suma;

/**
 *
 * @author Adrian
 */
public class Main {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
    }

    public int suma(int a, int b){
        return a + b;
    }
}
```

Unit Testing – Example 1 (2)

Projects | Files | Services

- Suma
  - Source Packages
    - suma
      - Main.java
  - Test Packages
  - Libraries
  - Test Libraries

suma - Navigator

Members View

- Main
  - main(String[] a
  - suma(int a, int

Start Page | Main.java *

```
/*
 * To change this template, choose Tools | Template
 open the template in the editor.

 suma;

 hor Adrian

 class Main {

 @param args the command line ar

 lic static void main(String[] a
 // TODO code application logic

 lic int suma(int a, int b){
 return a + b;
```

Open	
Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Compile File	F9
Run File	Shift+F6
Debug File	Ctrl+Shift+F5
Profile File	
Test File	Ctrl+F6
Add	
Delete	Delete
Save As Template...	
Find Usages	Alt+F7
Refactor	▸
BeanInfo Editor...	
File Members	Ctrl+F12
File Hierarchy	Alt+F12
Local History	▸
Tools	▸
Properties	

Add to Favorites	
Create JUnit Tests	Ctrl+Shift+
Analyze Javadoc	
Add to Palette...	

Select JUnit Version

Select JUnit version for which the created test skeletons should be created:

- JUnit 3.x
- JUnit 4.x

Create Tests

Class to Test: suma.Main

Class Name: suma.MainTest

Location: Test Packages

Code Generation

Method Access Levels
- [x] Public
- [x] Protected
- [x] Package Private

Generated Code
- [x] Test Initializer
- [x] Test Finalizer
- [x] Default Method Bodies

Generated Comments
- [x] Javadoc Comments
- [x] Source Code Hints

OK | Cancel | Help

Unit Testing – Example 1 (3)

Suma - NetBeans IDE 6.7.1

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects | Files | Services

- Suma
  - Source Packages
    - suma
      - Main.java
  - Test Packages
    - suma
      - MainTest.java
  - Libraries
  - Test Libraries

Navigator

Members View

- MainTest
  - MainTest()
  - setUp()
  - setUpClass()
  - tearDown()
  - tearDownClass()
  - testMain()
  - testSuma()

Start Page | Main.java * | MainTest.java

```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */

package suma;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Adrian
 */
public class MainTest {

    public MainTest() {
    }

    @BeforeClass
    public static void setUpClass() throws Exception {
    }

    @AfterClass
    public static void tearDownClass() throws Exception {
    }

    @Before
    public void setUp() {
    }

    @After
    public void tearDown() {
    }
```

Output | Tasks | Test Results

Unit Testing – Example 1 (4)

DE 6.7.1

ate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Services

Start Page | Main.java * | MainTest.java

```
    }

    /**
     * Test of suma method, of class Main.
     */
    @Test
    public void testSuma() {
        System.out.println("suma");
        int a = 0;
        int b = 0;
        Main instance = new Main();
        int expResult = 0;
        int result = instance.suma(a, b);
        assertEquals(expResult, result);
        // TODO review the generated test code and remove the default call
        fail("The test case is a prototype.");
    }

}
```

Unit Testing – Example 1 (5)

Projects | Files | Services

- Suma
  - Source Packages
    - suma
      - Main.java
  - Test Packages
    - suma
      - MainTest.java
  - Librarie
  - Test Li

Navigator

Members View

- MainTest
  - MainTe
  - setUp()
  - setUpC
  - tearDo
  - tearDo
  - testMai
  - testSur

Open
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Compile File F9
Run File Shift+F6
Debug File Ctrl+Shift+F5
Profile File
Test File Ctrl+F6
Add
Delete Delete
Save As Template...
Find Usages Alt+F7
Refactor
BeanInfo Editor...
File Members Ctrl+F12
File Hierarchy Alt+F12
Local History
Tools
Properties

Start Page | Main.java | MainTest.java

```
    }

    @Before
    public void setUp() {
    }

    @After
    public void tearDown() {
    }

    /**
     * Test of main method, of class Main.
     */
    @Test
    public void testMain() {
        System.out.println("main");
        String[] args = null;
        Main.main(args);
        // TODO review the generated test code and remove the default call to fail.
        fail("The test case is a prototype.");
    }

    /**
     * Test of suma method, of class Main.
     */
    @Test
    public void testSuma() {
```

Output - Suma (test) | Tasks | Test Results

0.0 %

No test passed, 2 tests failed.(0.145 s)

- suma.MainTest FAILED
  - testMain FAILED (at suma.MainTest.testMain(MainTest.java:49))
    - at suma.MainTest.testMain(MainTest.java:49)
  - testSuma FAILED (at suma.MainTest.testSuma(MainTest.java:65))
    - at suma.MainTest.testSuma(MainTest.java:65)

Test Results: main, suma

Unit Testing – Example 1 (6)

```
    @Test
    public void testSuma() {
        System.out.println("suma");
        int a = 0;
        int b = 0;
        Main instance = new Main();
        int expResult = 0;
        int result = instance.suma(a, b);
        assertEquals(expResult, result);
        // TODO review the generated test code and remove the default call to fail.
        //fail("The test case is a prototype.");
    }
}
```

Output - Suma (test) | **Tasks** | **Test Results**

50.0 %

1 test passed, 1 test failed.(0.019 s)

- suma.MainTest FAILED
  - testMain FAILED (at suma.MainTest.testMain(MainTest.java:49))
  - testSuma passed (0.0 s)

main
suma

Unit Testing – Example 2 (1)

BasicOperations.java | BasicOperationsTest.java

```java
package math;

public class BasicOperations {

    public int add(int x, int y){
        return x + y;
    }

    public int min(int x, int y){
        return x + y;
    }

    public int mul(int x, int y){
        return x * y;
    }

    public int div(int x, int y){
        return x / y;
    }

    /**
     * @param args
     */
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        BasicOperations bc = new BasicOperations();
        System.out.println(bc.add(3,5));
    }

}
```

Unit Testing – Example 2 (2)

Package Explorer | Hierarchy | JUnit

Finished after 0.019 seconds

Runs: 4/4 Errors: 0 Failures: 1

- test.BasicOperationsTest [Runner: JUnit 4] (0.004 s)
  - testAdd (0.000 s)
  - testMin (0.003 s)
  - testMul (0.000 s)
  - testDiv (0.001 s)

Failure Trace

```
java.lang.AssertionError: Result expected:<2> but was:<8>
at test.BasicOperationsTest.testMin(BasicOperationsTest.java:20)
```

BasicOperations.java | BasicOperationsTest.java

```java
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.div(4, 2));
    }
}
```

Unit Testing – Example 2 (3)

Package Explorer | Hierarchy | JUnit

Finished after 0.019 seconds

Runs: 4/4 Errors: 1 Failures: 0

- test.BasicOperationsTest [Runner: JUnit 4] (0.004 s)
  - testAdd (0.000 s)
  - testMin (0.000 s)
  - testMul (0.000 s)
  - testDiv (0.004 s)

Failure Trace

```
java.lang.ArithmeticException: / by zero
at math.BasicOperations.div(BasicOperations.java:18)
at test.BasicOperationsTest.testDiv(BasicOperationsTest.java:32)
```

BasicOperations.java | BasicOperationsTest.java

```java
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 0, bo.div(4, 0));
    }
}
```

Unit Testing – Example 2 (4)

Package Explorer | Hierarchy | JUnit

Finished after 0.016 seconds

Runs: 4/4 Errors: 0 Failures: 0

- test.BasicOperationsTest [Runner: JUnit 4] (0.000 s)
  - testAdd (0.000 s)
  - testMin (0.000 s)
  - testMul (0.000 s)
  - testDiv (0.000 s)

Failure Trace

BasicOperations.java | BasicOperationsTest.java

```java
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertTrue("Result", 8 == bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertFalse("Result", ! (3 != bo.min(5, 3)));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        if(bo.div(4, 2) == 3)
            fail("Incorrect result!");
    }
}
```

Code Coverage

- NetBeans – TikiOne JaCoCoverage:
- http://plugins.netbeans.org/plugin/48570/tikione-jacocoverage

- Java Code Coverage for Eclipse:
- http://www.eclemma.org/

- IntelliJ – Running with coverage:
- https://www.jetbrains.com/help/idea/2016.3/running-with-coverage.html

NetBeans – 1

JavaApplication1 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects | Files | Services

JavaApplication1

Source Pa
javaa
Ja
Test Pac
javaa
Ja
Libraries
Test Libr

JavaApplication1Te

Members

JavaApplica
JavaApp
setUp()
setUpCla
tearDow
tearDow
testMain
testPoz1
testSum
testSum

New
Build
Clean and Build
Clean
Generate Javadoc
Run
Debug
Profile
Test Alt+F6
Run Selenium Tests
Set Configuration
Open Required Projects
Close
Rename...
Move...
Copy...
Delete Delete
Find... Ctrl+F
Inspect and Transform...
Versioning
History
Run with JaCoCoverage
Test with JaCoCoverage
Reset coverage data

JavaApplication1.java | JavaApplication1Test.java

Source | History

```
    }

    @Test
    public void testSum2(){
        JavaApplication1 j1 = new JavaApplication1();
        assertEquals(-8,j1.Sum(-5,-3));
    }

    @Test
    public void testPoz1(){
        JavaApplication1 j1 = new JavaApplication1();
        assertEquals(8,j1.Poz(8));
    }

    /*@Test
    public void testPoz2(){
        JavaApplication1 j1 = new JavaApplication1();
        assertEquals(8,j1.Poz(-8));
    }*/
}
```

javaapplication1.JavaApplication1Test

Results | Output

JavaApplication1 (jacocoverage report) | JavaApplication1 (test)

```
main
Tests run: 4, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.041 sec

------------- Standard Output ---------------
main
------------- --------------- ---------------
```

74:8 INS

NetBeans – 2

JavaApplication1 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

JavaApplication1

Source

Test Pac

New

Build

Clean and Build

JavaApplication1.java JavaApplication1Test.java

Source History

```
}

@Test
public void testSum2(){
```

Adrian

JaCoCoverage analysis o

file:///C:/Users/Adrian/Documents/NetBeansProjects/JavaApplication1/.jacocoverage/report.html/index.html

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma)

Session

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
javaapplication1		80%		50%	1	5	1	7	0	4	0	1
Total	4 of 20	80%	1 of 2	50%	1	5	1	7	0	4	0	1

Created with JaCoCo 0.7.6.201602180

NetBeans – 3

JavaApplication1 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects | Files | Services

JavaApplication1

Source Pa

javaa

Ja

Test Pac

New

Build

Clean and Build

JavaApplication1.java | JavaApplication1Test.java

Source | History

```
}

@Test
public void testSum2(){
```

JaCoCoverage analysis of

file:///C:/Users/Adrian/Documents/NetBeansProjects/JavaApplication1/.jacocoverage/report.html/index.html

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma)

Session

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from

JavaApplication1

file:///C:/Users/Adrian/Documents/NetBeansProjects/JavaApplication1/.jacocoverage/report.html/javaapplication1/JavaApplica

JaCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma) > javaapplication1 > JavaApplication1

Session

JavaApplication1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Poz(int)		50%		50%	1	2	1	3	0	1
main(String[])		100%		n/a	0	1	0	2	0	1
Sum(int, int)		100%		n/a	0	1	0	1	0	1
JavaApplication1()		100%		n/a	0	1	0	1	0	1
Total	4 of 20	80%	1 of 2	50%	1	5	1	7	0	4

Created with JaCoCo 0.7.6.201602180

Using AI to Generate Testing Code

- Using AI Coding assistant (GitHub Copilot, Microsoft Copilot, ChatGPT)
- Prompt based Testing Code Generation (Claude)
- Custom AI models (fine tunned GPT models)
- *create a Java class for student with a method that verify the year of a student*
- *Develop a comprehensive suite of unit tests for the student. Write multiple test methods that cover a wide range of scenarios, including edge cases, exception handling, and data validation.*

https://github.com/copilot/

github.com/copilot/c/666f92ab-41d4-4b2d-a34a-b7bdbb4e8e30

Copilot

Type / to search

GPT-4o

create a Java class for student with a method that verify the year of a student

Here's a Java class for a `Student` with a method to verify the year of the student:

Student.java v3

```java
public class Student {
    private String name;
    private int year;

    public Student(String name, int year) {
```

This `Student` class has:

- Two private fields: `name` and `year`.
- A constructor to initialize these fields.
- Getter methods for `name` and `year`.
- A `verifyYear` method that checks if the student's year matches the expected year.
- A `main` method to demonstrate the u e of the `verifyYear` method.

Feel free to add more methods or modify it as needed!

Ask Copilot

Person.java | Professor.java | Student.java | StudentTest

v3 · 26 lines · 602 bytes

```java
public class Student {
    private String name;
    private int year;

    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    public String getName() {
        return name;
    }

    public int getYear() {
        return year;
    }

    public boolean verifyYear(int expectedYear) {
        return this.year == expectedYear;
    }

    public static void main(String[] args) {
        Student student = new Student("John Doe", 2023);
        System.out.println("Student Year Verification: " + student.verifyYear(2
    }
}
```

Use `Control + Shift + m` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page.

Type / to search

Person.java | Professor.java | Student.java | StudentTes

v3 · 69 lines · 2 KB

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StudentTest {

    @Test
    public void testVerifyYear_CorrectYear() {
        Student student = new Student("John Doe", 2023);
        assertTrue(student.verifyYear(2023), "The year should be 2023");
    }

    @Test
    public void testVerifyYear_IncorrectYear() {
        Student student = new Student("John Doe", 2023);
        assertFalse(student.verifyYear(2022), "The year should not be 2022");
    }

    @Test
    public void testVerifyYear_EdgeCaseYear() {
        Student student = new Student("Jane Doe", 0);
        assertTrue(student.verifyYear(0), "The year should be 0");
    }

    @Test
    public void testVerifyYear_NegativeYear() {
        Student student = new Student("Jane Doe", -1);
```

Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then tab to move to the next interactive element on the page.

Type / to search

Person.java | Professor.java | Student.java | StudentTes

v3 · 69 lines · 2 KB

```java
    @Test
    public void testGetName() {
        Student student = new Student("John Doe", 2023);
        assertEquals("John Doe", student.getName(), "Name should be John Doe");
    }

    @Test
    public void testGetYear() {
        Student student = new Student("John Doe", 2023);
        assertEquals(2023, student.getYear(), "Year should be 2023");
    }

    @Test
    public void testSetName() {
        Student student = new Student("John Doe", 2023);
        student.setName("Jane Doe");
        assertEquals("Jane Doe", student.getName(), "Name should be Jane Doe");
    }

    @Test
    public void testSetYear() {
        Student student = new Student("John Doe", 2023);
        student.setYear(2024);
        assertEquals(2024, student.getYear(), "Year should be 2024");
    }
}
```

Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then ta
interactive element on the page.

https://copilot.cloud.microsoft/

copilot.cloud.microsoft/?auth=2

Copilot
create a Java class for student with a method that...

New chat

create a Java class for student with a method that verify the year of a student

Copilot

Sure! Here's a simple Java class for a student with a method to verify the year of the student:

```
public class Student {
    private String name;
    private int year;

    // Constructor
    public Student(String name, int year) {
        this.name = name;
        this.year = year;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Getter for year
    public int getYear() {
        return year;
    }
```

Can you explain the test cases? | How to run these tests in an IDE? | What if I want to use Mockito? | View prompts

Ask me anything...

0 / 8000

Copilot

Agents

Get agents

Create an age

Recent chats

create a Java cla

Generate classes

See more

copilot.cloud.microsoft/?auth=2

Copilot

create a Java class for student with a method that...

New chat

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StudentTest {

    @Test
    public void testConstructorAndGetters() {
        Student student = new Student("John Doe", 2025);
        assertEquals("John Doe", student.getName());
        assertEquals(2025, student.getYear());
    }

    @Test
    public void testVerifyYear() {
        Student student = new Student("John Doe", 2025);
        assertTrue(student.verifyYear(2025));
        assertFalse(student.verifyYear(2024));
    }

    @Test
    public void testVerifyYearWithEdgeCases() {
        Student student = new Student("John Doe", Integer.MAX_VALUE);
        assertTrue(student.verifyYear(Integer.MAX_VALUE));
        assertFalse(student.verifyYear(Integer.MIN_VALUE));
    }
```

Can you explain the test cases? | How to run these tests in an IDE? | What if I want to use Mockito? | View prompts

Ask me anything...

0 / 8000

Conclusions

- Design Patterns
  - Definitions, Elements, Example, Classification
- JUnit Testing
- Code Coverage

Myths

- **Clients**
  - A general description of the objectives is sufficient to begin writing program
  - Requirements are constantly changing, but the software is flexible and can easy adapts
- **Developers**
  - Once the program is written and it is functional, our role has ended
  - Until the program doesn't work, we can not assess the quality
  - The only good product is the functional program
  - Software Engineering will create voluminous and unnecessary documentation and will cause delays

Design Patterns – Questions

- 1) Argue for the use of DP.
- 2) Come up with arguments not to use DP.
- Criticism: http://sourcemaking.com/design_patterns

Bibliografie

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)
- Ovidiu Gheorghieş, Curs 7 IP
- Adrian Iftene, Curs 9 TAIP: http://thor.info.uaic.ro/~adiftene/Scoala/2011/TAIP/Courses/TAIP09.pdf

Links

- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm
- Gang of Four: http://en.wikipedia.org/wiki/Gang_of_four
- JUnit in Eclipse: http://www.vogella.de/articles/JUnit/article.html
- JUnit in NetBeans: http://netbeans.org/kb/docs/java/junit-intro.html

Thank you!

For presence,
patience,
collaboration…